TASK-4

In a hospital management system, each patient record consist of Patient ID, Age, and Severity Score.
The hospital need patients for treatment based on following rules:

1. Patients must be sorted by severity Score in descending order.
2. If two patient have same they must be sorted in Ascending order.

Implement a merge sort algorithm using Divide & Conquer technique to sort patient records efficiently.

AIM: To implement the Merge sort algorithm using Divide and Conquer technique, sort a given set of elements in ascending or descending order based on user choice, and determine the time required to perform the Sorting.

ALGORITHM: 1. Start the program

2. Read the noof elements n.
3. Read n elements from the user and store them in array.
4. Ask the user to choose the sorting order.
   - 1 for Ascending
   - 2 for Descending
5. Call the merge sort () function with the selected order.
6. In mergesort ():
   - If the array has more than one element, divide it into two halves.
   - Recursively apply merge sort on the left and right subarrays.
   - Merge the two sorted subarrays using the merge() function
7. In merge():
   - compare elements from both subarrays
   - Arrange elements in ascending or descending order based on user choice.
8. Measure the execution time using clock().

9. Display the sorted elements & the time taken.

10. Stop the program.

PROGRAM :

```
# include <stdio.h>
# include <time.h>
void merge( int a[], int l, int m, int r, int desc) {
    int i, j, k;
    int n1 = m-l+1;
    int n2 = r-m;
  int L[n1], R[n2];
    for (i=0; i<n1 ; i++)
        L[i] = a[l+i];
    for (j=0 ; j<n2 ; j++)
        R[j] = a[m+1+j];
  i = j = 0 ;
    k = l ;
  while (i<n1 && j< n2) {
      if (! desc) {
        if( L[i] <= R[j])
          a[k++] = L[i++] ;
      else
          a[k++] = R[j++];
    } else {
        if( L[i] >= R[j])
          a[k++] = L [i++] ;
      else
          a[k++] = R[j++] ;
        }
      }
          while (i < n1)
          a[k++] = L[i++];
        while (j< n2)
          a[k++] = R[j++];
        }
```

```
Void mergeSort (int a[], int l, int r, int desc){
    if (l<r){
        int m = (l+r)/2;
        mergeSort(a, l, m, desc);
        mergeSort (a, m+1, r, desc);
        merge (a, l, m, r, desc);
        }
    }
        int main(){
        int n, i, choice;
        clock_t start, end;
        double time_taken;
Printf("Enter number of elements:");
        scanf ("%d", &n);
    int a[n];
    Printf("Enter %d elements:\n", n);
    for(i=0; i<n; i++)
        scanf ("%d", &a[i]);
Printf ("\nChoose order:\n");
Printf ("1. Ascending Order\n");
Printf ("2. Descending order\n");
Printf ("%d", &choice);
    start = clock();
    if( choice == 1)
        mergeSort(a, 0, n-1, 0);
    else
        mergeSort (a, 0, n-1, 1);
    end = clock();
    time_taken = (double)(end-start)
    Printf("\nSorted elements:\n");
    for (i=0; i<n; i++)
    Printf ("%d", a[i]);
```

OUTPUT : Enter a element : 6

Enter 6 elements.

456321 123456 654321 651234 345612 213456

choose order : Ascending order

Descending order

Enter your choice : descending order

654321 651234 456321 345612 213456 123456

```
Printf ("\n\nTime Taken : %f seconds\n", time_taken);
    return 0;
}
```

Analysis

Let n be the number of elements.

- Dividing the array :
  The array is divided into halves → $\log_2 n$ levels of recursion.
- Merging at each level :
  At each level, all n elements are compared and merged

Case	Time complexity	Reason
Best case	$O(n \log n)$	Devison & merging always occur
Average case	$O(n \log n)$	Balanced divison
worst case	$O(n \log n)$	Even if data is already sorted

Space complexity

Type	complexity
Auxiliary space	$O(n)$
Recursive stack space	$O(\log n)$
Total space complexity	$O(n)$

VEL TECH - CSE

EX NO.	4
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGN WITH DATE	10/2/26

RESULT : The merge sort algorithm was implemented Sucessfully and the given elements were sorted correctly in ascending or descending order. The execution time was measured and the algorithm was found to have a time complexity of $O(n \log n)$.